

Cory Hall: Digitally Deconstructed and Somewhat Reconstructed

May Liu
mayliujj@berkeley.edu

Brayton Lordianto
brayton.lordianto@berkeley.edu

Yue Huang
hyhy@berkeley.edu

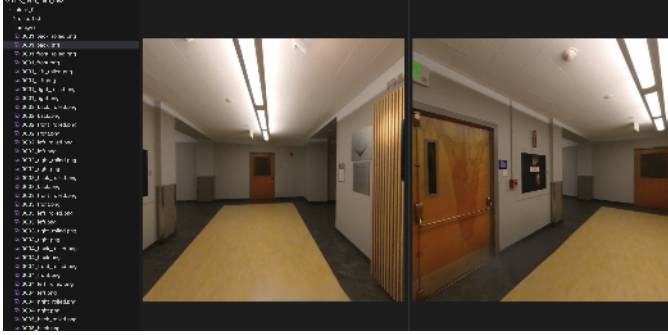


Fig. 1: Dataset 1: Cory Hall

Abstract—We explore a hybrid 3D reconstruction pipeline that combines structure-from-motion, monocular depth estimation, and neural rendering techniques to efficiently reconstruct complex real-world indoor environments. Using a large custom dataset collected from Cory Hall, we create 3D reconstructions of it using 2D Gaussian Splatting (2DGS), which results in interactive, real-time visualizations. To retrieve dense depth maps and accurate camera trajectories for reconstruction, we apply and evaluate two methods, namely MegaSaM and standard COLMAP. We also benchmark and evaluate these techniques alongside MaSt3R, a recent neural surface optimization method, highlighting the trade-offs between speed and geometric fidelity. We then discuss how we can retrieve acceptable mesh quality from the reconstructed results.

I. INTRODUCTION

3D scene reconstruction is a central problem in computer vision that involves recovering dense geometry and photometric properties of a scene from a set of 2D observations. With the growing demand for realistic digital twins, AR/VR environments, and robotic perception, recent methods have pushed the boundary in both accuracy and efficiency.

However, existing reconstructions using methods like 3D Gaussian Splatting are only accurate for Novel View Synthesis. In addition, there are limitations with many reconstruction pipelines, which is that they have assumptions based on scene rigidity, camera calibration, and curated input conditions.

In this project, we explore recent advances in structure-from-motion and image matching—specifically MegaSaM [1] and MAST3R [2]—and evaluate their performance on a real-world dataset collected using a drone in Cory Hall. We then extract a mesh from 2D Gaussian Splatting results and discuss our implemented post-processing mesh techniques tailored for indoor space reconstructions.



Fig. 2: Dataset 2: Cory Room. Raw Image Capture Prior to Inpainting and SAM Segmentation

This project is a branched extension of an existing capstone project in 2023-2024 and now in 2024-2025. The capstone team captured image data with a 360 degree RGB camera mounted on top to collect a continuous stream on images of that traversal (Fig. 1), applied preprocessing, used COLMAP, and created Novel View Synthesis (NVS) reconstructions of indoor scenes using 3D Gaussian Splatting [7]. On top of this project, we evaluate whether using COLMAP free methods like MegaSaM and Mast3r gives better rendering quality.

Existing methods struggle from the complexities of COLMAP, and our hypothesis is that better NVS reconstructions can be improved using COLMAP free methods. Additionally, understanding that meshes will continue to be important in the realm of digital reconstruction, we expand the scope from NVS results to mesh results and wish to explore whether the latest and greatest COLMAP free methods can improve these reconstructions. All techniques and work in this report was done solely by the writers of this work, unless indicated otherwise.

II. DATA INPAINTING

The dataset consists of 360-degree images captured by Stephen Lee from our capstone team using a drone-mounted camera (Fig. 2). To prepare the data for input, Shiran Yuan applied SAM-based segmentation to identify and mask out drone components. We then implemented and performed inpainting on the raw images to remove the masked regions and produce clean scene inputs suitable for reconstruction (Fig. 3).

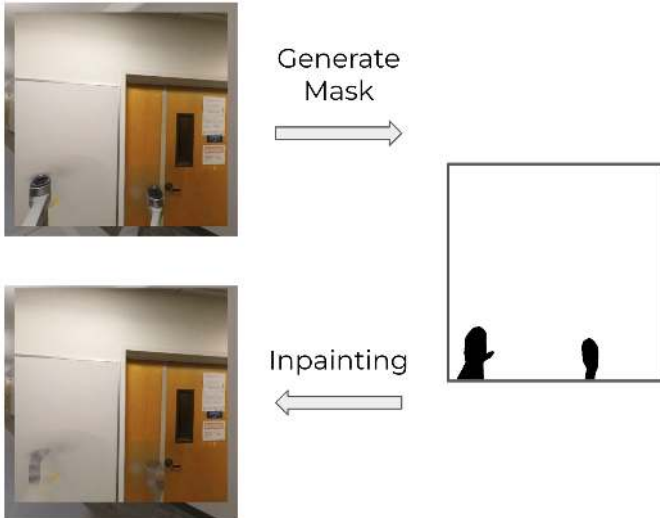


Fig. 3: Inpainting Applied

III. MEGASAM AND 2DGS

To address the limitations of traditional SfM pipelines in challenging indoor environments, we applied MegaSaM [1] to our custom dataset captured via drone in Cory Hall. MegaSaM offers a fast and robust SLAM-based pipeline that integrates learned monocular depth estimation with a factor graph optimization backend to recover dense depth maps and camera poses from image inputs. Its reliance on frame-to-frame geometric consistency, rather than feature matching across wide baselines, makes it particularly effective in scenes with texture repetition, low lighting, and constrained navigation spaces.

After obtaining the reconstructed data in .npz format, we developed a custom script to convert into .ply point cloud files for direct visualization and inspection. This script extracts RGB images, depth maps, camera intrinsics, and poses from the MegaSaM-generated archive, and reprojects the depth information into 3D space using the corresponding camera parameters. Each valid depth pixel is backprojected into a 3D point in the global coordinate frame and associated with its RGB color, producing a colored point cloud in the standard .ply format. These .ply outputs are visualized using Open3D, which provides interactive inspection tools and allows qualitative assessment of the reconstruction fidelity. Across multiple scenes, the MegaSaM-generated point clouds exhibit clear spatial layout, sharp structural boundaries, and consistent color information. Architectural features such as walls, hallways, staircases, and railings are all well preserved. Despite being generated from monocular depth estimation, the resulting point clouds show strong global consistency due to MegaSaM’s SLAM-based pose optimization.

Importantly, we observe that MegaSaM recovers a coherent global structure of the scene without the need for COLMAP’s sparse feature triangulation. The point clouds reflect an accurate understanding of scene geometry and layout, even in challenging lighting or texture-deprived areas, demonstrating the pipeline’s robustness. This step serves

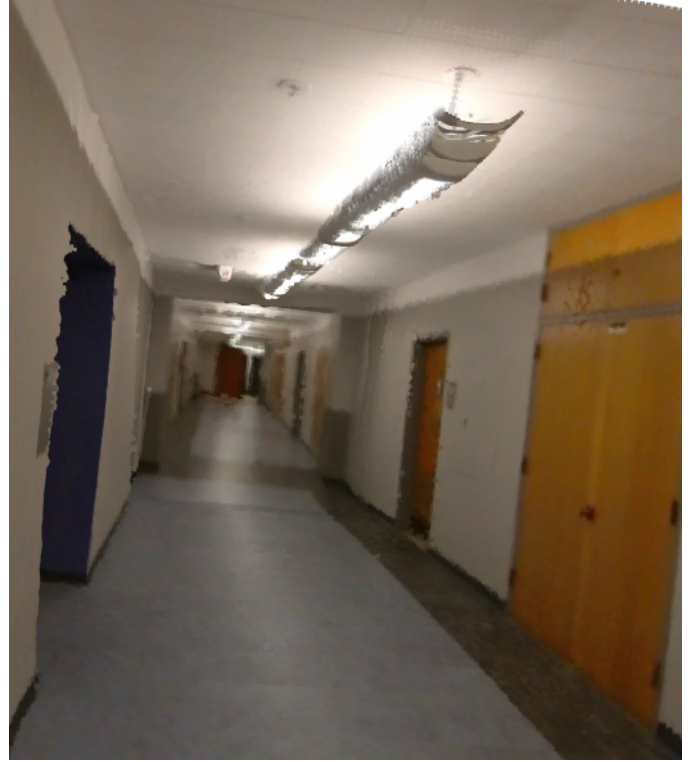


Fig. 4: Side View of MegaSaM’s Converted Point Cloud



Fig. 5: Top View of MegaSaM’s Converted Point Cloud

not only as an intermediate validation tool but also as a standalone output that can support downstream tasks such as mesh reconstruction or semantic annotation.

1) NerfStudio and MegaSaM: [4]

We attempted to integrate MegaSaM’s outputs with downstream rendering pipelines by converting MegaSaM’s NPZ format into Nerfstudio’s expected data structure. Nerfstudio had built in models for Gaussian splatting which we used, namely Splatfacto.

To convert the data, we first take the outputs of MegaSaM. MegaSaM outputs camera intrinsics as a 3×3 matrix and camera-to-world poses, as 4×4 transformation matrices. Our pipeline extracts focal lengths and principal points from the intrinsic matrix and reformats them according to Nerfstudio’s conventions.

RGB images from MegaSaM are saved as individual frame files in PNG format. We then create a transforms.json file that contains all camera parameters, image dimensions, and frame-specific transformation matrices. This file serves as the primary input for Nerfstudio’s training pipeline. Our results

from Nerfstudio are reasonable but it seems it does not work well for our pipeline (see Figure 6).

2) *Directly Using 2DGS on MegaSaM*: We attempted to feed our MegaSaM output directly to 2DGS, in hopes that the results would be better. Since 2DGS takes in COLMAP format files, we had to convert the NPZ files to COLMAP’s directory structure. To do this, we create binary file structures with correct coordinate systems to create the files `cameras.bin`, `images.bin`, `points3D.bin`, `points3D.ply`. We use mostly the same extractions as we do with our conversion to Nerfstudio format. A critical aspect of the conversion is transforming MegaSaM’s camera-to-world matrices to COLMAP’s quaternion and translation vector representation.

3) *Results and Discussion*: Since MegaSaM is optimized for sequential image inputs and leverages temporal consistency between adjacent frames, we restrict our input to front-facing images extracted from the drone footage. This ensures stable depth estimation and reliable pose tracking along the camera trajectory. However, this design choice introduces a view bias into the reconstruction: while the 2D Gaussian Splatting (2DGS) renderings appear highly accurate and visually coherent in the forward-facing direction of motion, the reconstructions degrade significantly when viewed from the opposite direction. In these back-facing views, we observe the presence of floaters—artifacts caused by occluded or poorly estimated geometry—due to the lack of multi-view coverage and depth supervision. This limitation highlights the importance of input diversity in monocular SLAM-based pipelines and motivates future extensions to incorporate multi-directional or panoramic capture strategies for more complete 3D reconstructions.

To better understand the trade-offs, we compare COLMAP-generated outputs as input to 2DGS. Unlike MegaSaM, COLMAP performs feature matching across a broader set of views, allowing it to reconstruct scenes from front, side, and back-facing images. As a result, 2DGS reconstructions based on COLMAP inputs exhibit improved completeness and visual quality, especially when rendering from viewpoints opposite the direction of camera motion. In these cases, floaters are significantly reduced, and the geometry is better defined due to the more balanced multi-view supervision. However, the major drawback of the COLMAP-based pipeline is its computational cost: feature extraction, matching, and sparse reconstruction typically take between 1–3 hours on standard hardware. In contrast, MegaSaM completes its dense depth and pose estimation in under 5 minutes, offering a compelling speed advantage for time-sensitive applications where view completeness can be traded off for efficiency.

IV. MAST3R

In addition to MegaSaM, we also tried an alternative COLMAP-free method that we wanted to evaluate against MegaSaM. We specifically try Mast3R, which is an extension for scalability of top of the DUST3R work [2]. The technique predicts the depth of every point in an image, which is

called point mapping, in the calibrated coordinates of another image, a fundamentally different idea from MegaSaM. However, upon running it, we find that it took 30 minutes to process the same number of images that it took MegaSaM to process, and its results were far more inconsistent than MegaSaM (see Table I).

V. MESH EXTRACTION AND POST-PROCESSING

Our approach uses 2D Gaussian Splatting (2DGS) developed by Huang et al. (2024) to convert our indoor scene from point-based representations into initial mesh structures. These results provide good mesh reconstructions because the Gaussians are in 2D and have encoded normals, which are particularly useful for constructing meshes. While these methods provide foundational geometry, the directly extracted meshes have several geometrical problems including:

- 1) Irregular surface characteristics, such as dipping floors, caused by the specular lighting reflected off of indoor floors.
- 2) Incomplete capture of fine structural details leading to several cracks and holes on floors and ceilings.
- 3) Numerous mesh discontinuities and artifacts, most that appear from floaters from applying Gaussian Splatting.

These limitations necessitated the development of comprehensive post-processing pipelines to transform these preliminary meshes into high-quality models suitable for downstream applications. Our pipeline is specialized to work on indoor reconstructed environments with the specific problems that we have described.

1) *Artifact Removal*: We find artifacts where we expect there to be floaters in the gaussian splatting. For instance, the blurry regions found in the left side of Figure 9 are caused by floaters, which are tiny nearly transparent gaussians that float in the air. After geometry reconstruction, this ends up turning into the right side image, where we have triangles populating the floaters.

To remove these irregularities, we observe that most of these irregularities are disconnected from the rest of the indoor space mesh, whereas most of the correct mesh reconstruction are connected with each other. This means that taking the largest connected component of the mesh would result in a near artifact-free result. To do this, we simply extract the vertices, indexed faces, and RGB colors of the mesh. We then iterate over every face to determine the connectivity of the mesh, and find the connected components with a count of how many vertices are in the component. and simply retrieve the largest one. The images in Figure 9 show the effectiveness of our artifact removal process across two example regions.

2) *Floor Normalization and Correction*: Due to the specular lighting reflected by floor surfaces in indoor scenes, the rendering interprets the floors as being "dipped" in these lit regions. In the mesh result, this results in incorrect surface elevations of the floors.

To address this problem, we first find the floor’s predicted Y coordinate through statistical analysis of the mesh’s vertical distribution. We can do this because we can identify that

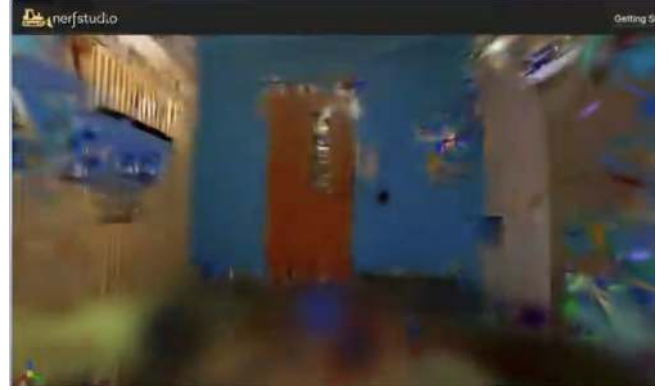
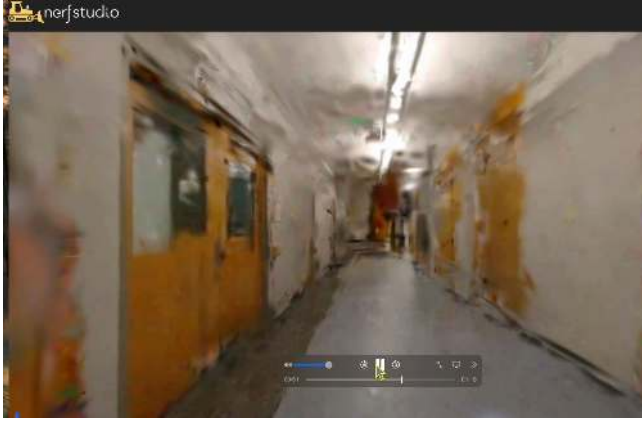


Fig. 6: Nerfstudio renderings using camera poses and depth priors from the MegaSaM pipeline.



(a) Side View



(b) Top View

TABLE I: MAST3R reconstruction



Fig. 7: MegaSaM 2DGS Render Along Camera Direction



Fig. 8: MegaSaM 2DGS Rendering from the Opposite Camera Direction

an indoor scene reconstruction would have most density in specifically the floors and ceilings. This follows the architectural principle that indoor spaces typically have continuous horizontal surfaces at consistent elevations. As expected, the histogram of vertices show bimodal characteristics, with significant density concentrations at the floor and ceiling planes (see Figure 12.) By applying Gaussian smoothing to the Y-

value histogram and identifying the lower-frequency peak, we can robustly determine the floor plane’s elevation despite local geometric distortions caused by specular artifacts.

Finally, we simply loop through all vertex positions and correct them to be capped at no less than the floor’s Y coordinate. The results of this floor normalization performs

very well qualitatively (see Figure 11).

3) *Hole Detection and Hole Filling*: A significant challenge of the extracted meshes was the presence of numerous holes, especially on the walls or floors. Holes can be defined in meshes as a loop, or a connected closed path, of only boundary edges. Boundary edges are edges that are adjacent to or only present in exactly one face (instead of two faces, like interior edges). For a only single block of Cory Hall, we identified 68,195 holes.

To fix holes, we implement a simplified algorithm that adds additional faces with existing vertices [3]. We first detect the boundary edges in every hole by looping over all the faces and identifying which edge is only called once. We then observe that, the same hole will contain another edge that contains the vertex in this edge. Therefore, if we start with edge $E = (u, v)$, we can find the next edge in the hole by looping over remaining boundary edges that contain vertex v . A full iteration until we reach the same initial vertex u allows us to retrieve all vertices in the hole.

After we identify an array of vertices for every hole, we can then fill the hole by triangulation. An appropriate method to do this is to do Delanuy triangulation to fill it, as this will create well-shaped triangles respecting local curvature. However, this triangulation method does not work well in 3D and it is necessary to project it in 2D. This is even more appropriate for our use case because our primary hole filling goal is to fix small sized cracks or large holes on the floors/walls, which means that most of these holes are close to a common plane (the floor/wall plane!). This means that this method, which creates flat patches, will respect the planar nature of the actual objects.

To get this best fit plane, we first get the singular values of the vertices of the hole to it's centroid. Each singular value is then the direction of greatest variance. If we consider floor holes, the largest variance would be in the X and Z values, as the Y value would be mostly close to the ground truth floor Y value. Then we project each vertex to the best fit plane, and we now have a 2D problem where we can simply run the Delunay algorithm to retrieve the faces we add to our faces list.

Our optimized algorithm completes this extensive filling operation for the 68,195 holes in just 71 seconds, and we have found that it scales linearly with the number of holes. Figure 10 show the results of this post-processing algorithm.

VI. RESULTS

Our experiments demonstrate that MegaSaM offers the fastest reconstruction performance among the evaluated methods (Mast3R, COLMAP). On our Cory Hall dataset, MegaSaM completes camera tracking and depth estimation within minutes, making it highly suitable for applications requiring fast turnaround or interactive feedback. This speed advantage is especially valuable in large-scale indoor environments where traditional methods become computationally expensive.

Qualitatively, the point clouds produced by MegaSaM are noticeably more structured but more dense than those from

Mast3R. While Mast3R applies neural surface optimization to recover geometry, it often suffers from noisy or incomplete outputs in textureless regions. In contrast, MegaSaM produces dense and visually consistent point clouds directly from monocular depth, thanks to its SLAM-based refinement process.

However, this performance comes with certain trade-offs. MegaSaM assumes a sequential image stream with smooth camera motion, which limits its ability to recover full 360° scene geometry. Areas not directly observed along the camera path, such as rear-facing surfaces or occluded regions, tend to be underrepresented or contain artifacts. This makes MegaSaM more suitable for trajectory-aligned reconstructions rather than complete scene capture.

In contrast, COLMAP, while significantly slower—taking 1–3 hours for sparse reconstruction—makes fewer assumptions about camera motion or image order. Its general-purpose structure-from-motion pipeline performs well on unordered image sets and produces more complete scene geometry, particularly in multi-view or non-linear capture scenarios. As a result, COLMAP remains a strong choice for applications requiring full-scene coverage, despite its higher computational cost.

We can also retrieve good meshes from 2DGS running on top of COLMAP results. However, these meshes have many artifacts and inconsistent structures. We prepare the meshes usage for downstream applications by applying a post processing method that considers the geometry of common indoor scene reconstructions.

VII. LINKS

Github Implementation
Website For Video Results

REFERENCES

- [1] Z. Li, R. Tucker, F. Cole, Q. Wang, L. Jin, V. Ye, A. Kanazawa, A. Holynski, and N. Snavely, "MegaSaM: Accurate, Fast, and Robust Structure and Motion from Casual Dynamic Videos," arXiv preprint arXiv:2401.12588, 2024.
- [2] V. Leroy, Y. Cabon, and J. Revaud, "Grounding Image Matching in 3D with MAST3R," arXiv preprint arXiv:2406.09756, 2024.
- [3] A. Brunton, S. Wuhler, C. Shu, P. Bose and E. D. Demaine, "Filling holes in triangular meshes by curve unfolding," 2009 IEEE International Conference on Shape Modeling and Applications, Beijing, China, 2009, pp. 66-72, doi: 10.1109/SMI.2009.5170165. keywords: Filling;Geometry;Solid modeling;Shape;USA Councils;Geometrical optics;Laser modes;Computational Geometry;Object Modeling;Hole Filling;Curve Unfolding.
- [4] M. Tancik, B. Mildenhall, S. Fridovich-Keil, A. Yu, E. Tretschk, D. Singh, et al., "Nerfstudio: A modular framework for neural radiance field development," ACM Trans. Graph., vol. 42, no. 4, pp. 1–11, 2023.
- [5] B. Huang, Z. Yu, A. Chen, A. Geiger, and S. Gao, "2D Gaussian Splatting for Geometrically Accurate Radiance Fields," arXiv preprint arXiv:2401.08512, 2024.
- [6] Nerfstudio Project, "gsplat: 2D Gaussian Splatting for radiance field rendering," GitHub repository, <https://github.com/nerfstudio-project/gspat> (accessed Apr. 11, 2025).
- [7] Y. Chen, C. Zhang, J. Wang, X. Gao, and A. Zakhor, "Scalable Indoor Novel-View Synthesis using Drone-Captured 360 Imagery with 3D Gaussian Splatting," arXiv preprint arXiv:2403.13606, Mar. 2024. [Online]. Available: <https://arxiv.org/abs/2403.13606>



(a) Original mesh with artifacts (Example 1)



(b) After artifact removal (Example 1)



(c) Original mesh with artifacts (Example 2)



(d) After artifact removal, and additional processing mentioned in later subsections (Example 2)

Fig. 9: Artifact removal process showing before and after states for two example regions. The filtering algorithm successfully identifies and removes irregularities while preserving the underlying structure.



(a) Mesh with visible holes after artifact removal



(b) After hole filling process (Example 1)



(c) Another region with holes (Example 2). Notice the small cracks on the ceiling lights and the patches on the floor.



(d) After hole filling process (Example 2). Notice the disappearance of the cracks on the ceiling and the filling in of thin primitives on the small patches of the floor.

Fig. 10: Hole filling process demonstrating the detection and repair of mesh discontinuities. The algorithm identifies boundary edges and generates appropriate fill geometry while maintaining mesh consistency.

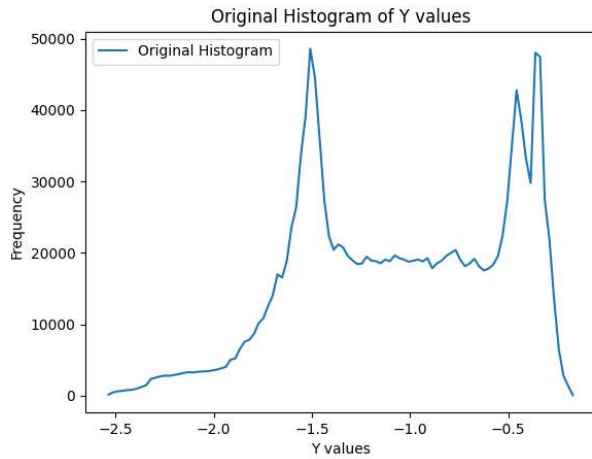


(a) Mesh with dipping floor regions

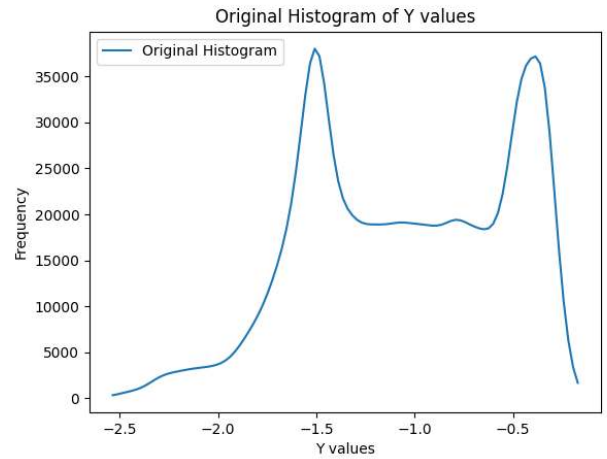


(b) After artifact removal and preventing floor dipping

Fig. 11: Floor correction process showing the identification and resolution of dipping floor regions. The left image shows problematic areas where floors incorrectly dip below expected elevation levels, while the right image demonstrates the corrected mesh with properly aligned floor surfaces after processing.



(a) Histogram of mesh polygon Y-values



(b) Histogram after Gaussian Smoothing

Fig. 12: Statistical verification of floor plane coordinates through histogram analysis. The before histogram shows a raw histogram of y values, while the right shows a smoothed histogram of these values.

Aspect	COLMAP	Mast3R	MegaSaM
Speed	Slow (~2hrs for our dataset of 300 images)	Slow (~30mins for 100 images)	Fast (~5mins for 100 images, 30mins for 300)
Reconstruction Coverage	Best for full 360° and all angles	Partial, optimized for view synthesis	Limited angled reconstructions, view-dependent
Point Cloud Quality	Moderate, noisy in dense mode	Noisy and less consistent	Clean and consistent
Best Fit For	Full scene reconstruction	Neural view synthesis research	Fast camera trajectory + point cloud

Fig. 13: Comparison of COLMAP, Mast3R, MegaSaM